

4.2 Handout for Pseudo-code convention

The conventions must be followed by your pseudo-code. Please note that the code should be **clear** and **easy to understand**.

1. Give a valid name for the pseudo-code procedure, specify the input variables' names and types, specify the output type. (See sample code for insertion sort at the end).
2. Use proper **Indentation** for every statement in a block structure.
3. For a flow control statements use **if-else**. Always end an **if** statement with an **end-if**. Both *if*, *else* and *end-if* should be aligned vertically in same line.

Ex:

```
if (conditional expression)
    statements (see the indentation)
else
    statements
end-if
```

4. Use $\leftarrow$ operator for assignment statements.

Ex:

```
i  $\leftarrow$  j
n  $\leftarrow$  2 to length[A]
```

5. Array elements can be represented by specifying the array name followed by the index in square brackets. For example, **A[i]** indicates the **ith** element of the array **A**.
6. For looping or iteration use **for** or **while** statements. Always end a **for** loop with **end-for** and a **while** with **end-while**.

7. The conditional expression of **for** or **while** can be written as shown in rule (4). You can separate two or more conditions with an **and**.

Sample pseudo-code for insertion sort using the above conventions

Algorithm *binarySearch*(*searchKey*, *A*, *n*)

Input *array A of n integers, searchKey to be searched in A*

Output *the index of the searchKey in A if found, n if not found*

```
1: lowerBound  $\leftarrow$  0
2: upperBound  $\leftarrow$  n - 1
3: while true do
4:   curIn  $\leftarrow$  (lowerBound + upperBound)/2
5:   if A[curIn] = searchKey then
6:     return curIn
7:   else if lowerBound > upperBound then
8:     return n
9:   else
10:    if A[curIn] > searchKey then
11:      upperBound  $\leftarrow$  curIn - 1
12:    else
13:      lowerBound  $\leftarrow$  curIn + 1
14:    end if
15:  end if
16: end while
```

Algorithm *INSERTION-SORT*(*A*)

Input *array A[0...n-1] with n items in random order*

Output *array A whose items are in ascending order*

```
1: for j  $\leftarrow$  2 to length[A] do
2:   key  $\leftarrow$  A[j]
```

```
3:   $i \leftarrow j - 1$ 
4:  while  $i > 0$  and  $A[i] < key$  do
5:     $A[i + 1] \leftarrow A[i]$ 
6:     $i \leftarrow i - 1$ 
7:  end while
8:   $A[i + 1] \leftarrow key$ 
9: end for
```